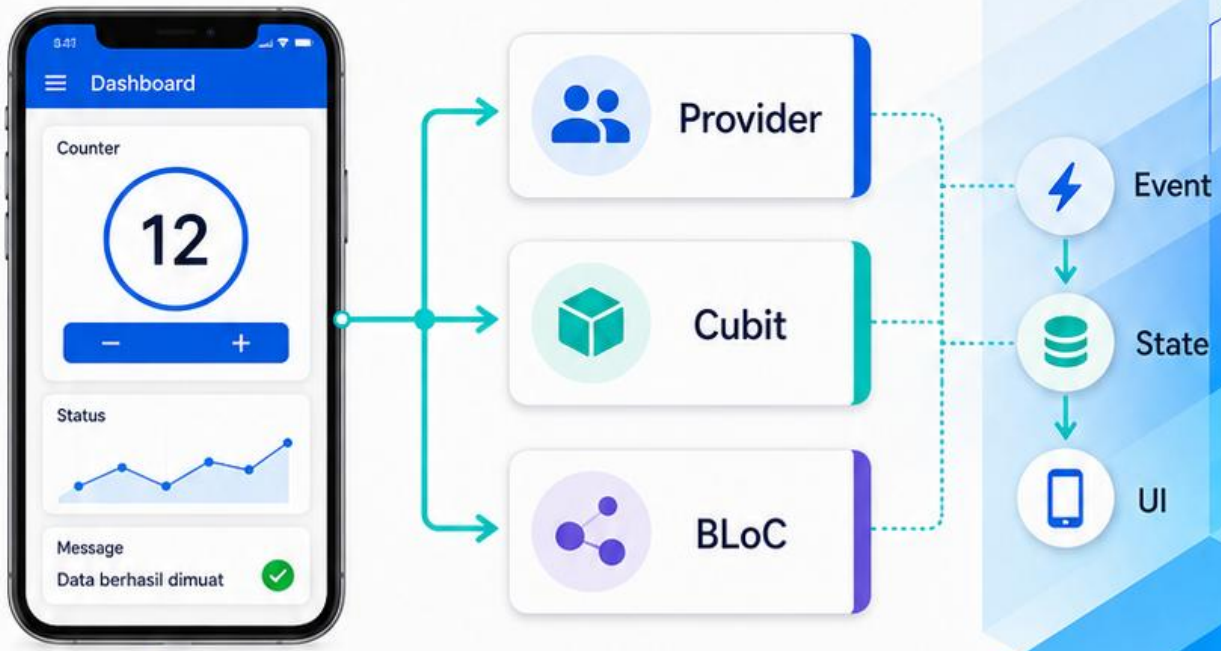
MODUL PRAKTIIKUM 05

STATE MANAGEMENT PADA APLIKASI FLUTTER

Provider • Cubit • BLoC



Mata Kuliah	Pemrograman Perangkat Bergerak Lanjut
Modul	V – State Management, Provider, Cubit, dan BLoC
Tahun	2026

Modul praktikum ini digunakan sebagai panduan kegiatan praktikum mahasiswa.

DAFTAR ISI

A. Tujuan Pembelajaran 3
B. Tugas Pendahuluan 3
C. Dasar Teori 3
a. Mengapa Perlu *State* Management yang Lebih Terstruktur 3
b. Mengenal Kembali *Provider* (*ChangeNotifier*) 4
c. Mengenal *Cubit* pada *flutter_bloc* 5
d. Mengenal *BLoC* (Business Logic Component) 5
e. Perbandingan *Provider*, *Cubit*, dan *BLoC* 6
f. Kapan Menggunakan *Provider*, *Cubit*, atau *BLoC* 7
g. Struktur Folder dan Alur Data pada *Cubit*/*BLoC* 8
D. Alat dan Bahan 8
a. Perangkat Keras (Hardware) 8
b. Perangkat Lunak (Software) 9
E. Materi Praktikum 9
a. Menyiapkan Proyek Praktikum 9
b. Membuat *CartCubit* untuk Mengelola Keranjang Belanja 9
c. Menghubungkan *Cubit* dengan *BlocProvider* dan *BlocBuilder* 11
d. Membuat *CartBloc* dengan *Event* dan *State* 17
e. Menghubungkan *Bloc* dengan *BlocProvider* dan *BlocBuilder* 19
f. Menampilkan Efek Samping dengan *BlocListener* dan *BlocConsumer* 25
g. Membandingkan *Provider*, *Cubit*, dan *BLoC* pada Layar yang Sama 27
F. Materi Praktikum dengan Prompt 28
a. Menyiapkan Proyek Praktikum dengan Bantuan AI 28
b. Membuat *CartCubit* dengan Bantuan AI 29
c. Menghubungkan *Cubit* dengan Bantuan AI 30
d. Membuat *CartBloc* dengan Bantuan AI 32
e. Menghubungkan *Bloc* dengan Bantuan AI 33
f. Menampilkan Efek Samping dengan Bantuan AI 34
g. Membandingkan Ketiga Pendekatan dengan Bantuan AI 35
h. Menguji dan Memperbaiki Galat dengan Bantuan AI 36
i. Mengevaluasi Jawaban AI 37
G. Tugas Praktikum 37
H. Format Pengumpulan Tugas 40

A. Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Menjelaskan perbedaan konsep antara *Provider*, *Cubit*, dan *BLoC* dalam pengelolaan *state* pada aplikasi Flutter.
- Mengimplementasikan *Cubit* menggunakan paket *flutter_bloc* untuk mengelola *state* sederhana tanpa *event* eksplisit.
- Mengimplementasikan *BLoC* dengan *Event* dan *State* untuk mengelola *state* secara terstruktur dan mudah diuji.
- Menghubungkan *Cubit* dan *Bloc* ke antarmuka pengguna menggunakan *BlocProvider*, *BlocBuilder*, *BlocListener*, dan *BlocConsumer*.
- Membandingkan kompleksitas, jumlah *boilerplate*, dan kemudahan penggunaan antara *Provider*, *Cubit*, dan *BLoC* pada studi kasus yang sama.
- Menentukan pendekatan *state* management yang tepat (*Provider*, *Cubit*, atau *BLoC*) berdasarkan kebutuhan dan skala aplikasi.
- Menggabungkan beberapa pendekatan *state* management dalam satu aplikasi multi-halaman dengan navigasi.

B. Tugas Pendahuluan

1. Jelaskan mengapa aplikasi Flutter yang semakin besar memerlukan state management yang lebih terstruktur! Sebutkan masalah yang dapat muncul jika seluruh state hanya dikelola dengan setState() atau satu ChangeNotifier yang terlalu besar!
2. Apa yang dimaksud dengan Cubit pada paket flutter_bloc? Jelaskan cara Cubit mengubah state melalui method dan emit(), serta perbedaannya dengan ChangeNotifier pada Provider yang menggunakan notifyListeners()!
3. Jelaskan konsep BLoC (Business Logic Component) beserta peran Event dan State di dalamnya! Mengapa BLoC dianggap lebih mudah dilacak dan diuji dibandingkan Cubit?
4. Jelaskan perbedaan fungsi BlocProvider, BlocBuilder, BlocListener, dan BlocConsumer!
5. Bandingkan Provider, Cubit, dan BLoC berdasarkan kompleksitas, jumlah boilerplate, dan skala aplikasi yang cocok!

C. Dasar Teori

Bagian ini membahas konsep dasar tiga pendekatan *state* management pada Flutter, yaitu *Provider*, *Cubit*, dan *BLoC*, perbedaan mendasar di antara ketiganya, serta pedoman praktis untuk menentukan pendekatan mana yang paling sesuai dengan kebutuhan aplikasi.

a. Mengapa Perlu *State* Management yang Lebih Terstruktur

Pada modul sebelumnya, Saudara telah menggunakan *setState*() untuk *state* lokal dan *Provider* (*ChangeNotifier*) untuk *state* yang dibagikan ke banyak *widget*. Pada aplikasi berskala kecil, kedua pendekatan tersebut sudah cukup memadai. Namun, ketika aplikasi berkembang menjadi lebih besar, dengan banyak halaman dan logic bisnis yang kompleks, pendekatan tersebut mulai menunjukkan sejumlah keterbatasan.

- Logic bisnis mudah tercampur dengan kode antarmuka (UI) sehingga sulit diuji secara terpisah dari *widget*.
- Tidak ada aturan baku untuk membedakan antara "*state*" yang harus terus ditampilkan pada layar dan "*event*" sekali pakai, misalnya menampilkan SnackBar atau berpindah halaman setelah sebuah aksi berhasil.
- Riwayat dan alur perubahan *state* menjadi sulit dilacak seiring bertambahnya jumlah *ChangeNotifier* pada aplikasi.
- Pengujian (unit test) terhadap logic bisnis menjadi lebih sulit karena logic tersebut bercampur dengan pemanggilan *notifyListeners*() dan bergantung pada *BuildContext*.

Untuk mengatasi keterbatasan tersebut, komunitas Flutter mengembangkan pustaka *flutter_bloc* yang menyediakan dua pola pengelolaan *state*, yaitu *Cubit* dan *BLoC*. Keduanya dibangun di atas prinsip yang sama dengan *Provider*, yaitu memisahkan *state* dari *widget*, tetapi dengan aturan yang lebih ketat, alur data yang lebih jelas, dan struktur yang lebih mudah diuji.

b. Mengenal Kembali *Provider* (*ChangeNotifier*)

Provider adalah pustaka *state* management yang memanfaatkan mekanisme *InheritedWidget* bawaan Flutter dengan cara yang lebih sederhana. Kelas *state* pada *Provider* biasanya diturunkan dari *ChangeNotifier*, dan setiap kali data berubah, method *notifyListeners*() dipanggil agar seluruh *widget* yang berlangganan (*subscribe*) melakukan *build* ulang.

- *State* disimpan pada satu kelas yang extends *ChangeNotifier*.
- Perubahan *state* diberitahukan secara manual melalui pemanggilan *notifyListeners*().
- *Widget* diberi akses ke *state* menggunakan *ChangeNotifierProvider* di bagian atas *widget* tree.
- *Widget* membaca *state* menggunakan context.watch<T>(), *Consumer*<T>, atau memanggil method melalui context.read<T>().

Contoh CartProvider dengan ChangeNotifier

```
class CartProvider extends ChangeNotifier {
  final List<String> _items = [];
  List<String> get items => _items;

  void addItem(String item) {
    _items.add(item);
    notifyListeners();
  }

  void removeItem(String item) {
    _items.remove(item);
    notifyListeners();
  }
}
```

Pola *CartProvider* di atas sudah dipelajari pada Modul 3. Pada modul ini, logic yang sama akan diimplementasikan ulang menggunakan *Cubit* dan *BLoC* sebagai pembanding langsung, sehingga perbedaan strukturnya terlihat jelas.

c. Mengenal *Cubit* pada *flutter_bloc*

Cubit adalah kelas dasar yang disediakan oleh pustaka *flutter_bloc* untuk mengelola *state* tanpa perlu mendefinisikan *event* secara eksplisit. *Cubit* dapat dianggap sebagai versi yang lebih terstruktur dari *ChangeNotifier*: sama-sama menyimpan satu buah *state*, tetapi perubahan *state* dilakukan dengan mengeluarkan (*emit*) objek *state* yang baru secara utuh, bukan dengan mengubah properti yang sudah ada.

- *Cubit* dibuat dengan cara extends *Cubit*<*State*>, dengan *State* berupa tipe data apa pun (bisa int, bool, List, atau class kustom) yang merepresentasikan seluruh kondisi saat ini.
- Perubahan *state* dilakukan dengan memanggil *emit*(stateBaru) di dalam method milik *Cubit*, bukan dengan mengubah variabel lalu memanggil *notifyListeners*().
- *Cubit* tidak memiliki konsep *Event* terpisah; setiap method pada *Cubit* langsung memicu *emit*() ketika method tersebut dipanggil.
- *Cubit* paling cocok untuk logic yang sederhana dan langsung, misalnya *counter*, *toggle* status, atau daftar sederhana seperti keranjang belanja.

Contoh CartCubit

```
class CartCubit extends Cubit<List<String>> {
  CartCubit() : super([]);

  void addItem(String item) {
    emit([...state, item]);
  }

  void removeItem(String item) {
    emit(state.where((e) => e != item).toList());
  }
}
```

Perhatikan perbedaannya dengan *CartProvider*: *CartCubit* tidak memanggil *notifyListeners*(), melainkan *emit*(), yang secara otomatis memberi tahu seluruh *widget* yang mendengarkan. *State* juga tidak diubah secara langsung (mutasi), melainkan digantikan seluruhnya dengan objek baru, gaya ini disebut *state immutable*, dan menjadi praktik yang direkomendasikan agar perubahan *state* mudah dilacak dan diuji.

d. Mengenal *BLoC* (Business Logic Component)

BLoC adalah pola yang lebih ketat dibandingkan *Cubit*. Selain *State*, *BLoC* juga mendefinisikan *Event* sebagai representasi dari "apa yang terjadi" (misalnya pengguna menekan tombol tambah), sedangkan *State* tetap merepresentasikan "apa yang seharusnya ditampilkan". Alur datanya adalah: antarmuka mengirim *Event* → *Bloc* menerima *Event* pada sebuah *handler* → *Bloc* menjalankan logic bisnis → *Bloc* mengeluarkan (*emit*) *State* baru → antarmuka membangun ulang tampilan berdasarkan *State* tersebut.

- *BLoC* dibuat dengan cara extends *Bloc*<*Event*, *State*>, sehingga wajib mendefinisikan dua tipe data sekaligus: *Event* dan *State*.

- Setiap jenis *Event* ditangani oleh sebuah *handler* yang didaftarkan dengan on<TipeEvent>((*event*, *emit*) {...}); perubahan *state* tetap dilakukan melalui *emit*(), sama seperti pada *Cubit*.
- Karena setiap *Event* berupa objek yang eksplisit, riwayat "apa yang memicu perubahan" menjadi mudah dilacak, sehingga sangat membantu untuk *logging*, *debugging*, dan pengujian.
- *BLoC* lebih cocok digunakan pada logic bisnis yang kompleks, melibatkan banyak sumber input (tombol, respons API, *timer*, dan lain-lain), atau pada aplikasi yang menuntut pengujian yang ketat.

CartState, dan CartBloc

```
abstract class CartEvent {}

class AddItem extends CartEvent {
  final String item;
  AddItem(this.item);
}

class RemoveItem extends CartEvent {
  final String item;
  RemoveItem(this.item);
}

class CartState {
  final List<String> items;
  CartState(this.items);
}

class CartBloc extends Bloc<CartEvent, CartState> {
  CartBloc() : super(CartState([])) {
    on<AddItem>((event, emit) {
      emit(CartState([...state.items, event.item]));
    });
    on<RemoveItem>((event, emit) {
      emit(CartState(state.items.where((e) => e !=
event.item).toList()));
    });
  }
}
```

Dibandingkan dengan *CartCubit*, *CartBloc* memisahkan secara tegas antara "aksi apa yang terjadi" (AddItem, RemoveItem) dan "bagaimana meresponsnya" (*handler* on<...>). Pemisahan ini membuat kode *BLoC* sedikit lebih panjang (*verbose*) dibandingkan *Cubit*, tetapi memberikan struktur yang lebih jelas ketika logic bisnis aplikasi semakin kompleks.

e. Perbandingan *Provider*, *Cubit*, dan *BLoC*

Tabel 5.1 berikut merangkum perbedaan utama antara *Provider*, *Cubit*, dan *BLoC* agar karakteristik masing-masing dapat dilihat secara berdampingan.

Tabel 5.1 Perbandingan Provider, Cubit, dan BLoC

Aspek	*Provider* (*ChangeNotifier*)	*Cubit*	*BLoC*
Konsep dasar	extends *ChangeNotifier*, memanggil *notifyListeners*()	extends *Cubit<State>*, memanggil *emit*()	extends *Bloc<Event, State>*, menangani *Event* lalu memanggil *emit*()
Pemicu perubahan	Method dipanggil langsung dari *widget*	Method dipanggil langsung dari *widget*	*Event* dikirim dengan add(), lalu ditangani oleh handler on<*Event*>
Jumlah kode (*boilerplate*)	Sedikit	Sedikit hingga sedang	Sedang hingga banyak (perlu kelas *Event* dan *State* terpisah)
Kemudahan dipelajari pemula	Mudah	Mudah, konsepnya mirip *Provider*	Sedang, perlu memahami konsep *Event* dan Stream
Kemudahan pengujian (testing)	Sedang	Baik	Sangat baik karena *Event* dan *State* terpisah dengan jelas
Paling cocok untuk	*State* sederhana yang dibagikan ke beberapa *widget*	Logic sederhana dan langsung, misalnya counter atau toggle	Logic bisnis kompleks dengan banyak sumber aksi/input
Widget bantuan *flutter_bloc*	*Consumer*, *Provider*.of, context.watch/read	*BlocProvider*, *BlocBuilder*, *BlocListener*, *BlocConsumer*	*BlocProvider*, *BlocBuilder*, *BlocListener*, *BlocConsumer*
Skala aplikasi yang disarankan	Kecil hingga menengah	Kecil hingga menengah	Menengah hingga besar

Secara umum, *Cubit* dapat dipandang sebagai versi *Provider* yang lebih terstruktur dan lebih mudah diuji, sedangkan *BLoC* adalah versi *Cubit* yang lebih ketat dengan menambahkan konsep *Event*. Ketiganya berasal dari filosofi yang sama, yaitu memisahkan *state* dari *widget* tampilan, sehingga mahasiswa yang telah memahami *Provider* akan lebih mudah memahami *Cubit* maupun *BLoC*.

f. Kapan Menggunakan *Provider*, *Cubit*, atau *BLoC*

Bagi pemula, memilih di antara *Provider*, *Cubit*, atau *BLoC* dapat terasa membingungkan. Panduan sederhana berikut dapat dijadikan pedoman awal dalam menentukan pendekatan yang tepat.

- Gunakan *Provider* apabila *state* cukup sederhana dan hanya perlu dibagikan (di-*share*) ke beberapa *widget*, tanpa memerlukan pemisahan *Event* yang tegas. *Provider* juga sangat baik digunakan sebagai *dependency injection*, yaitu menyediakan objek seperti service atau *repository* ke *widget-widget* di bawahnya.
- Gunakan *Cubit* apabila logic bisnis tergolong sederhana dan langsung, misalnya *counter*, *toggle* tema, atau *filter* daftar, dan tim menginginkan kode yang ringkas namun tetap lebih terstruktur serta lebih mudah diuji dibandingkan *Provider*.

- Gunakan *BLoC* apabila logic bisnis kompleks dengan banyak sumber aksi (tombol, hasil panggilan API, *timer*, input pengguna lain), aplikasi berskala besar yang dikerjakan oleh banyak anggota tim dan membutuhkan riwayat aksi yang jelas untuk *debugging*, atau proyek yang menuntut pengujian otomatis (automated *testing*) secara menyeluruh.

Sebagai aturan praktis: jika masih ragu, mulailah dengan *Cubit*. *Cubit* memiliki kurva belajar yang mirip dengan *Provider*, tetapi sudah mengikuti struktur *flutter_bloc* sehingga aplikasi dapat dengan mudah ditingkatkan (*upgrade*) menjadi *BLoC* ketika logic bisnisnya bertambah kompleks di kemudian hari.

g. Struktur Folder dan Alur Data pada *Cubit*/*BLoC*

Pada praktik yang direkomendasikan, kode *Cubit*/*Bloc* dipisahkan dari kode antarmuka melalui struktur folder tertentu, misalnya sebagai berikut.

```
lib/
  cubit/          (atau bloc/)
    cart_cubit.dart
    cart_event.dart   (khusus BLoC)
    cart_state.dart
  screens/
    cart_page.dart
```

- Folder *cubit*/ atau *bloc*/ berisi seluruh logic bisnis: definisi *State*, *Event* (khusus *BLoC*), dan kelas *Cubit* atau *Bloc* itu sendiri.
- Folder screens/ hanya berisi *widget* yang menampilkan data dan mengirim aksi, yaitu memanggil method pada *Cubit* atau menambahkan *Event* ke *Bloc*, tanpa logic bisnis di dalamnya.
- Pemisahan ini memudahkan pengujian karena logic bisnis pada *Cubit*/*Bloc* dapat diuji tanpa perlu menjalankan *widget* sama sekali.

Secara ringkas, alur data pada *Cubit* adalah: *widget* memanggil method pada *Cubit* → *Cubit* memanggil *emit*(stateBaru) → *BlocBuilder* membangun ulang tampilan berdasarkan *state* tersebut. Sementara itu, alur data pada *BLoC* sedikit lebih panjang: *widget* mengirim *Event* dengan *bloc*.add(*event*) → *Bloc* menangani *Event* tersebut pada *handler* on<*Event*> → *Bloc* memanggil *emit*(stateBaru) → *BlocBuilder* atau *BlocConsumer* membangun ulang tampilan.

D. Alat dan Bahan

a. Perangkat Keras (Hardware)

Perangkat keras yang digunakan pada praktikum ini sama seperti modul-modul sebelumnya, yaitu:

- Laptop atau PC dengan RAM minimal 8 GB untuk menjalankan Android Studio/VS Code beserta emulator Flutter dengan lancar, 16 GB lebih disarankan.
- Smartphone Android atau iOS (opsional) untuk menguji aplikasi pada perangkat fisik.
- Koneksi internet yang stabil untuk mengunduh paket *flutter_bloc* dan dependensi lainnya.

b. Perangkat Lunak (Software)

Perangkat lunak yang dibutuhkan pada praktikum ini meliputi:

- Flutter SDK versi terbaru yang sudah terpasang dan teruji pada modul-modul sebelumnya.
- Android Studio atau Visual Studio Code beserta plugin Flutter dan Dart.
- Emulator Android/iOS, atau perangkat fisik yang telah diaktifkan mode pengembang (developer mode).
- Paket *flutter_bloc*, yaitu pustaka resmi yang menyediakan kelas *Cubit*, *Bloc*, serta *widget BlocProvider*, *BlocBuilder*, *BlocListener*, dan *BlocConsumer*.
- Paket *equatable* (opsional), yang membantu membandingkan dua objek *State* secara isi (value), bukan hanya berdasarkan referensi objeknya.

E. Materi Praktikum

Bagian ini membangun ulang aplikasi Kantin Kampus dari Modul 3 sebanyak dua kali,: proyek pertama menggunakan *Cubit*, proyek kedua menggunakan *BLoC*. Kedua proyek meniru persis tampilan dan alur Modul 3 (halaman *Produk* dan halaman Keranjang dengan navigasi bawah), hanya cara pengelolaan *state*-nya yang berbeda. Untuk versi *Provider*, proyek Modul 3 yang sudah ada dipakai sebagai pembanding. Setiap proyek diselesaikan dan diuji sampai berjalan sempurna sebelum melanjutkan ke proyek berikutnya.

a. Menyiapkan Proyek Praktikum

Sebelum memulai, pastikan proyek Kantin Kampus dari Modul 3 (versi *Provider*) masih tersimpan dan dapat dijalankan, karena proyek tersebut akan dipakai sebagai pembanding pada Bagian g. Buat proyek baru untuk versi *Cubit* dengan menjalankan perintah pada Kode 5.4.

```
flutter create kantin_kampus_cubit
cd kantin_kampus_cubit
flutter pub add flutter_bloc
```

- Di dalam proyek kantin_kampus_*cubit*, buat tiga folder di dalam lib/: models/, *cubit*/, dan pages/

b. Membuat *CartCubit* untuk Mengelola Keranjang Belanja

Bekerja pada proyek kantin_kampus_*cubit*. Kelas *Produk* dan *ItemKeranjang* pada Modul 3 dipakai apa adanya (hanya kelas *KeranjangModel* yang tidak dipakai, karena logic-nya dipindahkan ke *KeranjangCubit*).

- Buat file lib/models/*produk*.dart, lalu salin program di bawah ini.

```
import 'package:flutter/foundation.dart';

@immutable
class Produk {
  const Produk({
    required this.nama,
    required this.harga,
    required this.ikon,
  });
```

```
  final String nama;
  final int harga;
  final String ikon;
}

class ItemKeranjang {
  const ItemKeranjang({required this.produk, this.jumlah =
1});

  final Produk produk;
  final int jumlah;
}
```

- Buat file lib/*cubit*/keranjang_*cubit*.dart, lalu tulis kode dibawah ini. *KeranjangCubit* menyimpan *state* berupa List<*ItemKeranjang*>, menggantikan Map<String, *ItemKeranjang*> pada *KeranjangModel*, tetapi dengan hasil akhir yang sama: satu *produk* yang sama hanya muncul sekali di keranjang, dengan jumlah yang bertambah.

lib/cubit/keranjang_cubit.dart

```
import 'package:flutter_bloc/flutter_bloc.dart';

import '../models/produk.dart';

class KeranjangCubit extends Cubit<List<ItemKeranjang>> {
  KeranjangCubit() : super([]);

  void tambah(Produk produk) {
    final daftar = List<ItemKeranjang>.from(state);
    final indeks = daftar.indexWhere((item) => item.produk.nama == produk.nama);

    if (indeks == -1) {
      daftar.add(ItemKeranjang(produk: produk));
    } else {
      daftar[indeks] = ItemKeranjang(
        produk: produk,
        jumlah: daftar[indeks].jumlah + 1,
      );
    }

    emit(daftar);
  }

  void kurangi(Produk produk) {
    final daftar = List<ItemKeranjang>.from(state);
    final indeks = daftar.indexWhere((item) => item.produk.nama == produk.nama);
    if (indeks == -1) return;

    if (daftar[indeks].jumlah == 1) {
      daftar.removeAt(indeks);
    } else {
      daftar[indeks] = ItemKeranjang(
```

```
        produk: produk,
        jumlah: daftar[indeks].jumlah - 1,
      );
    }

    emit(daftar);
  }

  void kosongkan() {
    emit([]);
  }
}
```

- Perhatikan bahwa method tambah(), kurangi(), dan kosongkan() melakukan hal yang persis sama dengan method sejenis pada *KeranjangModel* Modul 3 (mencari *produk* yang sama, menambah/mengurangi jumlah, atau menghapus dari daftar). Perbedaannya hanya pada baris terakhir: *KeranjangModel* memanggil *notifyListeners*() setelah mengubah data secara langsung, sedangkan *KeranjangCubit* membuat daftar baru lalu mengirimkannya dengan *emit*().

c. Menghubungkan *Cubit* dengan *BlocProvider* dan *BlocBuilder*

Halaman *HalamanProduk* dan *HalamanKeranjang* dibuat mengikuti tampilan Modul 3 persis, hanya sumber datanya diganti dari *KeranjangModel* menjadi *KeranjangCubit*.

- Buat file lib/pages/halaman_*produk*.dart, lalu tulis kode dibawah ini. Bandingkan dengan lib/pages/halaman_*produk*.dart pada Modul 3, hanya baris context.read<*KeranjangModel*>() yang berubah menjadi context.read<*KeranjangCubit*>().

lib/pages/halaman_produk.dart (versi Cubit)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/keranjang_cubit.dart';
import '../models/produk.dart';

class HalamanProduk extends StatelessWidget {
  const HalamanProduk({super.key});

  static const daftarProduk = [
    Produk(nama: 'Nasi Goreng', harga: 15000, ikon: '🍛'),
    Produk(nama: 'Mie Ayam', harga: 13000, ikon: '🍜'),
    Produk(nama: 'Es Teh', harga: 5000, ikon: '🥤'),
  ];

  String rupiah(int nilai) => 'Rp${nilai.toString()}';

  @override
  Widget build(BuildContext context) {
    return ListView.separated(
```

```
      padding: const EdgeInsets.all(16),
      itemCount: daftarProduk.length,
      separatorBuilder: (context, indeks) => const SizedBox(height: 8),
      itemBuilder: (context, indeks) {
        final produk = daftarProduk[indeks];

        return Card(
          child: ListTile(
            leading: Text(produk.ikon, style: const TextStyle(fontSize: 32)),
            title: Text(produk.nama),
            subtitle: Text(rupiah(produk.harga)),
            trailing: FilledButton(
              onPressed: () {
                context.read<KeranjangCubit>().tambah(produk);
              },
              child: const Text('Tambah'),
            ),
          ),
        );
      },
    );
  }
}
```

- Buat file lib/pages/halaman_keranjang.dart, lalu tulis kode dibawah ini. Bandingkan dengan lib/pages/halaman_keranjang.dart pada Modul 3 — *Consumer<KeranjangModel>* diganti menjadi *BlocBuilder<KeranjangCubit*, List<*ItemKeranjang*>>, dan keranjang.items dibaca langsung sebagai parameter items pada builder.

lib/pages/halaman_keranjang.dart (versi Cubit)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/keranjang_cubit.dart';
import '../models/produk.dart';

class HalamanKeranjang extends StatelessWidget {
  const HalamanKeranjang({super.key});

  String rupiah(int nilai) => 'Rp${nilai.toString()}';

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<KeranjangCubit, List<ItemKeranjang>>(
      builder: (context, items) {
        if (items.isEmpty) {
          return const Center(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
```

```
          Icon(Icons.shopping_cart_outlined, size: 64),
          SizedBox(height: 12),
          Text('Keranjang masih kosong'),
        ],
      ),
    );
  }

  final totalHarga = items.fold<int>(
    0,
    (total, item) => total + (item.produk.harga * item.jumlah),
  );

  return Column(
    children: [
      Expanded(
        child: ListView.separated(
          padding: const EdgeInsets.all(16),
          itemCount: items.length,
          separatorBuilder: (context, indeks) => const Divider(),
          itemBuilder: (context, indeks) {
            final item = items[indeks];

            return ListTile(
              title: Text(item.produk.nama),
              subtitle: Text('${item.jumlah} x ${rupiah(item.produk.harga)}'),
              trailing: Row(
                mainAxisSize: MainAxisSize.min,
                children: [
                  IconButton(
                    onPressed: () =>
                        context.read<KeranjangCubit>().kurangi(item.produk),
                    icon: const Icon(Icons.remove_circle_outline),
                  ),
                  Text('${item.jumlah}'),
                  IconButton(
                    onPressed: () =>
                        context.read<KeranjangCubit>().tambah(item.produk),
                    icon: const Icon(Icons.add_circle_outline),
                  ),
                ],
              ),
            );
          },
        ),
      ),
      SafeArea(
        top: false,
        child: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
```

```
              children: [
                Text(
                  'Total: ${rupiah(totalHarga)}',
                  style: Theme.of(context).textTheme.titleLarge,
                ),
                const SizedBox(height: 8),
                OutlinedButton.icon(
                  onPressed: () => context.read<KeranjangCubit>().kosongkan(),
                  icon: const Icon(Icons.delete_outline),
                  label: const Text('Kosongkan Keranjang'),
                ),
              ],
            ),
          ),
        ),
      ],
    );
  },
 );
 }
}
```

- Ganti seluruh isi lib/main.dart. Struktur ini identik dengan lib/main.dart pada Modul 3 (*KerangkaUtama* dengan *NavigationBar* antara *Produk* dan Keranjang), hanya *ChangeNotifierProvider* diganti *BlocProvider* dan *KeranjangModel* diganti *KeranjangCubit*.

lib/main.dart (proyek kantin_kampus_cubit)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'cubit/keranjang_cubit.dart';
import 'pages/halaman_keranjang.dart';
import 'pages/halaman_produk.dart';

void main() {
 runApp(
  BlocProvider(
   create: (context) => KeranjangCubit(),
   child: const AplikasiKantin(),
  ),
 );
}

class AplikasiKantin extends StatelessWidget {
 const AplikasiKantin({super.key});

 @override
 Widget build(BuildContext context) {
  return MaterialApp(
   debugShowCheckedModeBanner: false,
```

```
      title: 'Kantin Kampus (Cubit)',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
        useMaterial3: true,
      ),
      home: const KerangkaUtama(),
    );
  }
}

class KerangkaUtama extends StatefulWidget {
  const KerangkaUtama({super.key});

  @override
  State<KerangkaUtama> createState() => _KerangkaUtamaState();
}

class _KerangkaUtamaState extends State<KerangkaUtama> {
  int _indeks = 0;

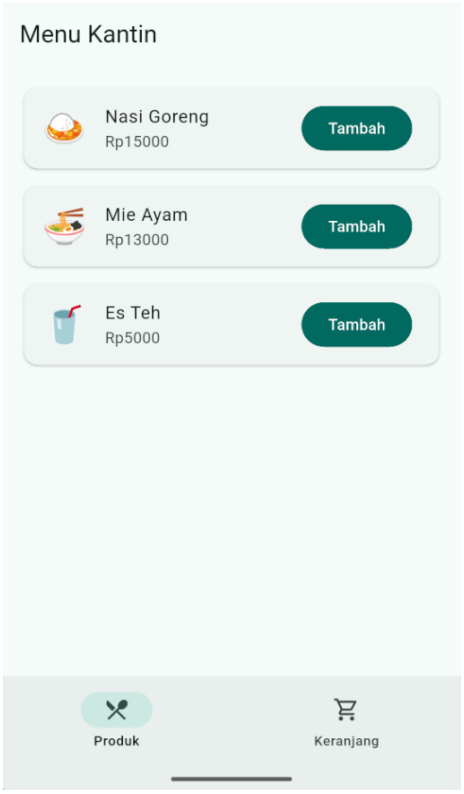
  static const _halaman = [
    HalamanProduk(),
    HalamanKeranjang(),
  ];

  @override
  Widget build(BuildContext context) {
    final totalItem = context.select<KeranjangCubit, int>(
      (cubit) => cubit.state.fold(0, (total, item) => total + item.jumlah),
    );

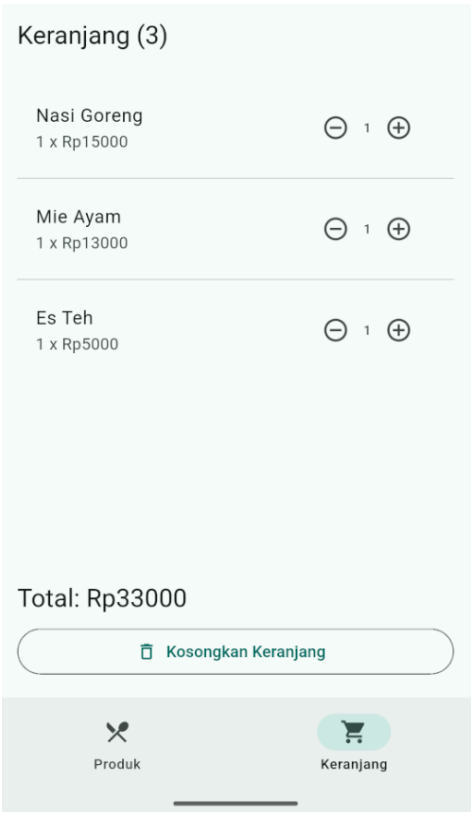
    return Scaffold(
      appBar: AppBar(
        title: Text(_indeks == 0 ? 'Menu Kantin' : 'Keranjang
($totalItem)'),
      ),
      body: IndexedStack(
        index: _indeks,
        children: _halaman,
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _indeks,
        onDestinationSelected: (nilaiBaru) {
          setState(() {
            _indeks = nilaiBaru;
          });
        },
        destinations: const [
          NavigationDestination(
            icon: Icon(Icons.restaurant_menu),
            label: 'Produk',
          ),
```

```
          NavigationDestination(
            icon: Icon(Icons.shopping_cart_outlined),
            selectedIcon: Icon(Icons.shopping_cart),
            label: 'Keranjang',
          ),
        ],
      ),
    );
  }
}
```

- Jalankan **flutter run**, lalu uji seluruh alur: tambah beberapa *produk* pada halaman *Produk*, pindah ke halaman Keranjang untuk melihat daftar dan total harga, kurangi/tambah jumlah item, dan coba tombol Kosongkan Keranjang. Pastikan semuanya berjalan sama seperti pada Modul 3 sebelum melanjutkan ke proyek *BLoC*.



Gambar 5.1 Tampilan halaman Produk pada proyek kantin_kampus_cubit



Gambar 5.2 Tampilan halaman Keranjang pada proyek kantin_kampus_cubit

d. Membuat *CartBloc* dengan *Event* dan *State*

- Buat proyek kedua untuk versi *BLoC*

```
flutter create kantin_kampus_bloc
cd kantin_kampus_bloc
flutter pub add flutter_bloc
```

- Di dalam proyek kantin_kampus_*bloc*, buat tiga folder di dalam lib/: models/, *bloc*/, dan pages/
- Buat file lib/models/*produk*.dart, lalu salin program di bawah ini.

```
import 'package:flutter/foundation.dart';

@immutable
class Produk {
  const Produk({
    required this.nama,
    required this.harga,
    required this.ikon,
  });

  final String nama;
  final int harga;
  final String ikon;
}

class ItemKeranjang {
  const ItemKeranjang({required this.produk, this.jumlah =
```

```
1});

  final Produk produk;
  final int jumlah;
}
```

- Buat file lib/*bloc*/keranjang_*bloc*.dart, lalu tulis kode berikut. Berbeda dengan *Cubit*, *BLoC* memerlukan dua kelas tambahan: *KeranjangEvent* (aksi yang dikirim dari tampilan) dan *KeranjangState* (data yang ditampilkan ke layar).

lib/bloc/keranjang_bloc.dart

```
import 'package:flutter_bloc/flutter_bloc.dart';

import '../models/produk.dart';

// Event: daftar aksi yang bisa dikirim ke KeranjangBloc
abstract class KeranjangEvent {}

class TambahProduk extends KeranjangEvent {
  TambahProduk(this.produk);
  final Produk produk;
}

class KurangiProduk extends KeranjangEvent {
  KurangiProduk(this.produk);
  final Produk produk;
}

class KosongkanKeranjang extends KeranjangEvent {}

// State: data yang ditampilkan ke layar
class KeranjangState {
  const KeranjangState(this.items);
  final List<ItemKeranjang> items;
}

// Bloc: menerima Event, lalu mengeluarkan State baru
class KeranjangBloc extends Bloc<KeranjangEvent,
KeranjangState> {
  KeranjangBloc() : super(const KeranjangState([])) {
    on<TambahProduk>((event, emit) {
      final daftar = List<ItemKeranjang>.from(state.items);
      final indeks = daftar.indexWhere((item) => item.produk.nama
== event.produk.nama);

      if (indeks == -1) {
        daftar.add(ItemKeranjang(produk: event.produk));
      } else {
        daftar[indeks] = ItemKeranjang(
          produk: event.produk,
          jumlah: daftar[indeks].jumlah + 1,
```

```
        );
      }

      emit(KeranjangState(daftar));
    });

    on<KurangiProduk>((event, emit) {
      final daftar = List<ItemKeranjang>.from(state.items);
      final indeks = daftar.indexWhere((item) => item.produk.nama
== event.produk.nama);
      if (indeks == -1) return;

      if (daftar[indeks].jumlah == 1) {
        daftar.removeAt(indeks);
      } else {
        daftar[indeks] = ItemKeranjang(
          produk: event.produk,
          jumlah: daftar[indeks].jumlah - 1,
        );
      }

      emit(KeranjangState(daftar));
    });

    on<KosongkanKeranjang>((event, emit) {
      emit(const KeranjangState([]));
    });
  }
}
```

- Perhatikan bahwa isi setiap handler (on<*TambahProduk*>, on<*KurangiProduk*>, on<*KosongkanKeranjang*>) persis sama dengan isi method tambah(), kurangi(), dan kosongkan() pada *KeranjangCubit*. Perbedaannya hanya pada cara aksi dikirim: *Cubit* memanggil method secara langsung, sedangkan *BLoC* harus melalui *Event*.

e. Menghubungkan *Bloc* dengan *BlocProvider* dan *BlocBuilder*

Halaman dibuat mengikuti struktur yang sama dengan proyek *Cubit*, hanya cara mengirim aksi dan tipe *state* yang berbeda.

- Buat file lib/pages/halaman_*produk*.dart, lalu tulis kode berikut.

lib/pages/halaman_produk.dart (versi BLoC)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/keranjang_bloc.dart';
import '../models/produk.dart';

class HalamanProduk extends StatelessWidget {
  const HalamanProduk({super.key});
```

```
  static const daftarProduk = [
    Produk(nama: 'Nasi Goreng', harga: 15000, ikon: '🍛'),
    Produk(nama: 'Mie Ayam', harga: 13000, ikon: '🍜'),
    Produk(nama: 'Es Teh', harga: 5000, ikon: '🧋'),
  ];

  String rupiah(int nilai) => 'Rp${nilai.toString()}';

  @override
  Widget build(BuildContext context) {
    return ListView.separated(
      padding: const EdgeInsets.all(16),
      itemCount: daftarProduk.length,
      separatorBuilder: (context, indeks) => const SizedBox(height:
8),
      itemBuilder: (context, indeks) {
        final produk = daftarProduk[indeks];

        return Card(
          child: ListTile(
            leading: Text(produk.ikon, style: const TextStyle(fontSize:
32)),
            title: Text(produk.nama),
            subtitle: Text(rupiah(produk.harga)),
            trailing: FilledButton(
              onPressed: () {

context.read<KeranjangBloc>().add(TambahProduk(produk));
              },
              child: const Text('Tambah'),
            ),
          ),
        );
      },
    );
  }
}
```

- Buat file lib/pages/halaman_keranjang.dart, lalu tulis kode berikut.

lib/pages/halaman_keranjang.dart (versi BLoC)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/keranjang_bloc.dart';
import '../models/produk.dart';

class HalamanKeranjang extends StatelessWidget {
  const HalamanKeranjang({super.key});

  String rupiah(int nilai) => 'Rp${nilai.toString()}';
```

```
 @override
 Widget build(BuildContext context) {
  return BlocBuilder<KeranjangBloc, KeranjangState>(
    builder: (context, state) {
     if (state.items.isEmpty) {
      return const Center(
       child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
         Icon(Icons.shopping_cart_outlined, size: 64),
         SizedBox(height: 12),
         Text('Keranjang masih kosong'),
        ],
       ),
      );
     }

     final totalHarga = state.items.fold<int>(
      0,
      (total, item) => total + (item.produk.harga * item.jumlah),
     );

     return Column(
      children: [
       Expanded(
        child: ListView.separated(
         padding: const EdgeInsets.all(16),
         itemCount: state.items.length,
         separatorBuilder: (context, indeks) => const Divider(),
         itemBuilder: (context, indeks) {
          final item = state.items[indeks];

          return ListTile(
           title: Text(item.produk.nama),
           subtitle: Text('${item.jumlah} x
${rupiah(item.produk.harga)}'),
           trailing: Row(
            mainAxisSize: MainAxisSize.min,
            children: [
             IconButton(
              onPressed: () => context
                  .read<KeranjangBloc>()
                  .add(KurangiProduk(item.produk)),
              icon: const Icon(Icons.remove_circle_outline),
             ),
             Text('${item.jumlah}'),
             IconButton(
              onPressed: () => context
                  .read<KeranjangBloc>()
                  .add(TambahProduk(item.produk)),
              icon: const Icon(Icons.add_circle_outline),
```

```
                    ),
                  ],
                ),
              );
            },
          ),
        ),
        SafeArea(
          top: false,
          child: Padding(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                Text(
                  'Total: ${rupiah(totalHarga)}',
                  style: Theme.of(context).textTheme.titleLarge,
                ),
                const SizedBox(height: 8),
                OutlinedButton.icon(
                  onPressed: () =>

context.read<KeranjangBloc>().add(KosongkanKeranjang()),
                  icon: const Icon(Icons.delete_outline),
                  label: const Text('Kosongkan Keranjang'),
                ),
              ],
            ),
          ),
        ),
      ],
    );
  },
 );
 }
}
```

- Ganti seluruh isi lib/main.dart.

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'bloc/keranjang_bloc.dart';
import 'pages/halaman_keranjang.dart';
import 'pages/halaman_produk.dart';

void main() {
 runApp(
  BlocProvider(
   create: (context) => KeranjangBloc(),
   child: const AplikasiKantin(),
  ),
```

```
  );
}

class AplikasiKantin extends StatelessWidget {
  const AplikasiKantin({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Kantin Kampus (BLoC)',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
        useMaterial3: true,
      ),
      home: const KerangkaUtama(),
    );
  }
}

class KerangkaUtama extends StatefulWidget {
  const KerangkaUtama({super.key});

  @override
  State<KerangkaUtama> createState() => _KerangkaUtamaState();
}

class _KerangkaUtamaState extends State<KerangkaUtama> {
  int _indeks = 0;

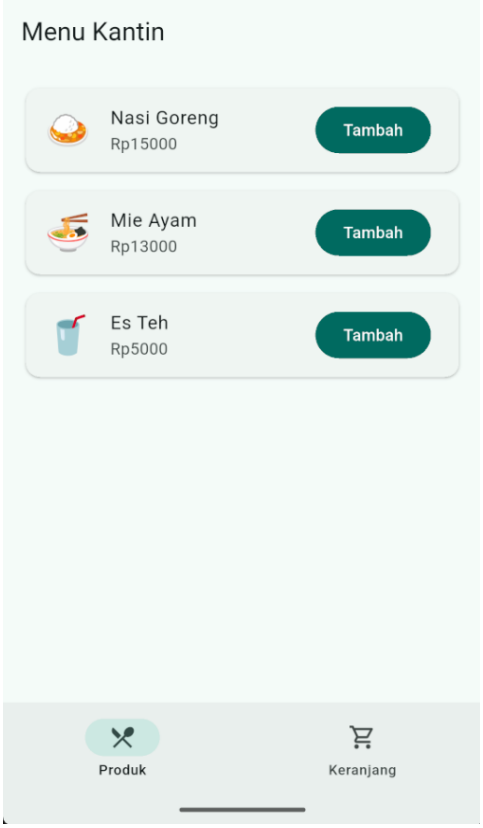
  static const _halaman = [
    HalamanProduk(),
    HalamanKeranjang(),
  ];

  @override
  Widget build(BuildContext context) {
    final totalItem = context.select<KeranjangBloc, int>(
      (bloc) => bloc.state.items.fold(0, (total, item) => total +
item.jumlah),
    );

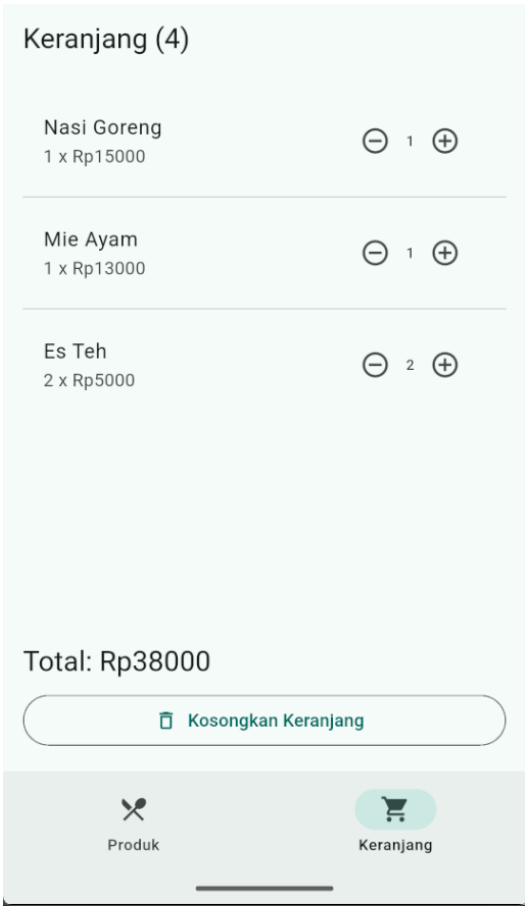
    return Scaffold(
      appBar: AppBar(
        title: Text(_indeks == 0 ? 'Menu Kantin' : 'Keranjang ($totalItem)'),
      ),
      body: IndexedStack(
        index: _indeks,
        children: _halaman,
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _indeks,
```

```
        onDestinationSelected: (nilaiBaru) {
          setState(() {
            _indeks = nilaiBaru;
          });
        },
        destinations: const [
          NavigationDestination(
            icon: Icon(Icons.restaurant_menu),
            label: 'Produk',
          ),
          NavigationDestination(
            icon: Icon(Icons.shopping_cart_outlined),
            selectedIcon: Icon(Icons.shopping_cart),
            label: 'Keranjang',
          ),
        ],
      ),
    );
  }
}
```

- Jalankan flutter run, lalu uji seluruh alur yang sama seperti pada proyek *Cubit*. Dari sisi tampilan, hasilnya identik dengan kantin_kampus_*cubit*; perbedaannya hanya pada cara mengirim aksi: *Cubit* memanggil method secara langsung, sedangkan *BLoC* selalu mengirim *Event* melalui add().



Gambar 5.3 Tampilan halaman Produk pada proyek kantin_kampus_bloc



Gambar 5.4 Tampilan halaman Keranjang pada proyek kantin_kampus_bloc

f. Menampilkan Efek Samping dengan *BlocListener* dan *BlocConsumer*

Sejauh ini, *BlocBuilder* dipakai untuk menampilkan data yang tersimpan pada *state*, misalnya daftar item di keranjang. Namun, ada jenis aksi lain yang sifatnya hanya perlu dijalankan satu kali ketika sebuah kejadian terjadi, bukan setiap kali tampilan digambar ulang, misalnya memunculkan SnackBar setiap kali *produk* berhasil ditambahkan. Aksi semacam ini disebut efek samping (*side effect*). Jika efek samping seperti ini diletakkan di dalam *BlocBuilder*, SnackBar berisiko muncul berulang-ulang setiap kali tampilan digambar ulang. Karena itu, *flutter_bloc* menyediakan dua *widget* khusus: *BlocListener* dan *BlocConsumer*.

- *BlocListener* hanya menjalankan efek samping (seperti menampilkan SnackBar) setiap kali *state* berubah, tanpa ikut menggambar *widget* apa pun ke layar.
- *BlocBuilder* hanya membangun ulang tampilan berdasarkan *state* terbaru, tidak cocok dipakai untuk efek samping.
- *BlocConsumer* menggabungkan keduanya dalam satu *widget*: parameter listener untuk efek samping, parameter builder untuk membangun tampilan.
- Kode di bawah adalah isi lengkap lib/pages/halaman_*produk*.dart pada proyek kantin_kampus_*bloc* setelah ditambahkan SnackBar menggunakan *BlocListener*, sehingga setiap kali tombol Tambah ditekan, muncul notifikasi singkat di bagian bawah layar.

lib/pages/halaman_produk.dart *dengan BlocListener*

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
```

```
import '../bloc/keranjang_bloc.dart';
import '../models/produk.dart';

class HalamanProduk extends StatelessWidget {
  const HalamanProduk({super.key});

  static const daftarProduk = [
    Produk(nama: 'Nasi Goreng', harga: 15000, ikon: '🍛'),
    Produk(nama: 'Mie Ayam', harga: 13000, ikon: '🍜'),
    Produk(nama: 'Es Teh', harga: 5000, ikon: '🧃'),
  ];

  String rupiah(int nilai) => 'Rp${nilai.toString()}';

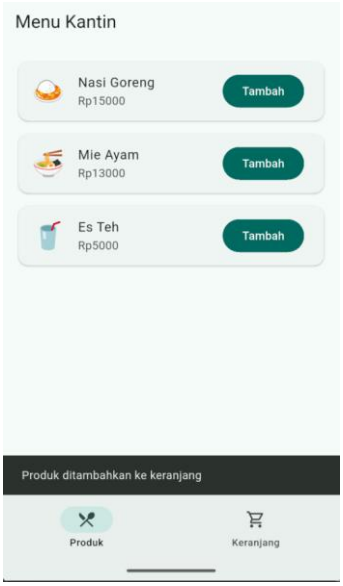
  @override
  Widget build(BuildContext context) {
    return BlocListener<KeranjangBloc, KeranjangState>(
      listener: (context, state) {
        if (state.items.isNotEmpty) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Produk ditambahkan ke
keranjang')),
          );
        }
      },
      child: ListView.separated(
        padding: const EdgeInsets.all(16),
        itemCount: daftarProduk.length,
        separatorBuilder: (context, indeks) => const
SizedBox(height: 8),
        itemBuilder: (context, indeks) {
          final produk = daftarProduk[indeks];

          return Card(
            child: ListTile(
              leading: Text(produk.ikon, style: const
TextStyle(fontSize: 32)),
              title: Text(produk.nama),
              subtitle: Text(rupiah(produk.harga)),
              trailing: FilledButton(
                onPressed: () {

context.read<KeranjangBloc>().add(TambahProduk(produk));
                },
                child: const Text('Tambah'),
              ),
            ),
          );
        },
      ),
    );
```

```
  }
}
```

- Perhatikan bahwa *BlocListener* membungkus (child:) tampilan ListView yang sudah ada, tanpa mengubah isi ListView itu sendiri. Listener hanya berjalan di latar belakang, menampilkan SnackBar setiap kali *state* berubah dan keranjang tidak kosong.
- Pola yang sama juga berlaku untuk *Cubit* pada proyek kantin_kampus_*cubit*, karena *Cubit* dan *Bloc* sama-sama merupakan turunan dari kelas dasar *BlocBase* pada *flutter_bloc*.



Gambar 5.5 Tampilan SnackBar yang muncul setelah produk ditambahkan

g. Membandingkan *Provider*, *Cubit*, dan *BLoC* pada Layar yang Sama

Berikut merupakan rangkuman perbandingan konkret antara *Provider*, *Cubit*, dan *BLoC,* sebagaimana Tabel 5.2 berikut.

Tabel 5.2 Perbandingan Konkret Provider, Cubit, dan BLoC

Aspek	***Provider (Modul 3)***	***Cubit (kantin_kampus_cubit)***	***BLoC (kantin_kampus_bloc)***
Kelas dasar yang di-extends	*ChangeNotifier*	*Cubit<List<ItemKeranjang>>*	*Bloc<KeranjangEvent, KeranjangState>*
Bentuk data (state)	*Map<String, ItemKeranjang> di dalam kelas, dibaca lewat getter items*	*State langsung berupa List<ItemKeranjang>*	*State berupa objek KeranjangState yang membungkus List<ItemKeranjang> items*
Cara mengubah data	*Mengubah _items secara langsung, lalu notifyListeners()*	*Membuat daftar baru, lalu emit(daftarBaru)*	*Mengirim Event (mis. TambahProduk) dengan add(), ditangani handler on<...>, lalu emit(stateBaru)*
Cara widget membaca data	*Consumer<KeranjangModel>*	*BlocBuilder<KeranjangCubit, List<ItemKeranjang>>*	*BlocBuilder<KeranjangBloc, KeranjangState>*

Aspek	*Provider (Modul 3)*	*Cubit (kantin_kampus_cubit)*	*BLoC (kantin_kampus_bloc)*
Widget penyedia (root aplikasi)	*ChangeNotifierProvider*	*BlocProvider*	*BlocProvider*
Jumlah kelas yang harus dibuat	*1 kelas (KeranjangModel)*	*1 kelas (KeranjangCubit)*	*3 kelas (KeranjangEvent, KeranjangState, KeranjangBloc)*
Aksi dari tombol Tambah	*context.read<KeranjangModel>().tambah(produk)*	*context.read<KeranjangCubit>().tambah(produk)*	*context.read<KeranjangBloc>().add(TambahProduk(produk))*

F. Materi Praktikum dengan Prompt

Bagian ini membangun ulang Materi Praktikum dengan bantuan asisten AI sebagai tools, untuk kedua proyek (*Cubit* dan *BLoC*). Hasil akhir yang dituju tetap sama dengan Bagian E, tetapi kode diperoleh melalui prompt bertahap agar mahasiswa tetap memahami setiap bagian, bukan sekadar menyalin jawaban AI.

Tabel 5.3 Unsur prompt yang digunakan pada praktikum berbantuan AI

Unsur prompt	*Penjelasan*
Peran	*Menentukan siapa AI dan tingkat pembacanya, misalnya asisten Flutter untuk mahasiswa pemula.*
Konteks	*Menjelaskan nama proyek, materi, berkas yang sudah ada, dan tahap yang sedang dikerjakan.*
Tugas	*Menyebutkan satu hasil yang jelas, misalnya menulis ulang satu berkas tertentu.*
Batasan	*Menjaga hasil tetap sesuai materi, paket yang diizinkan, nama kelas, dan struktur folder.*
Format keluaran	*Menentukan apakah AI harus memberi rencana, kode lengkap, bagian yang berubah, atau penjelasan.*
Verifikasi	*Meminta AI menjelaskan alasan kode dan memberi daftar pemeriksaan yang dapat diuji mahasiswa.*

a. Menyiapkan Proyek Praktikum dengan Bantuan AI

Tahap ini menghasilkan proyek baru yang terpisah dari praktikum manual, agar hasil kerja manual dan hasil berbantuan AI dapat dibandingkan.

- Buka terminal, lalu jalankan flutter create kantin_kampus_*cubit*_ai. Buka proyek pada jendela baru, lalu tambahkan dependensi *flutter_bloc* dengan flutter pub add *flutter_bloc*.
- Gunakan percakapan AI baru agar konteks dari proyek lain tidak ikut terbawa.
- Ketik prompt konteks awal berikut pada panel percakapan AI.

```
Kamu berperan sebagai asisten pemrograman Flutter untuk mahasiswa pemula.

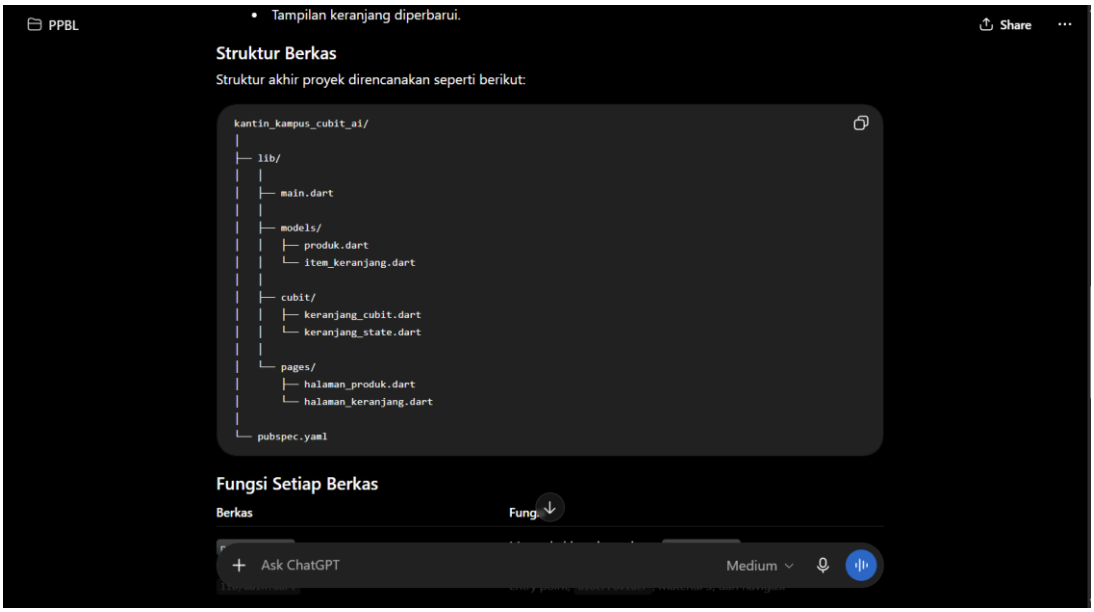
Konteks: saya membuat proyek bernama kantin_kampus_cubit_ai di Visual Studio Code.
Materi yang dipelajari adalah Cubit pada paket flutter_bloc.
```

Tujuan akhir: membantu saya membuat ulang aplikasi Kantin Kampus (sama seperti pada Modul 3) dengan halaman Produk dan halaman Keranjang, namun pengelolaan keranjangnya memakai KeranjangCubit, bukan ChangeNotifier.

Batasan:
1. Gunakan Flutter Material 3 dan hanya package:flutter/material.dart serta package:flutter_bloc/flutter_bloc.dart.
2. Pisahkan berkas model di lib/models, berkas Cubit di lib/cubit, dan berkas halaman di lib/pages.
3. Kerjakan bertahap. Jangan menulis semua kode sekaligus.
4. Setiap kode harus lengkap satu berkas, tidak boleh memakai tanda "..." untuk menghilangkan bagian kode.
5. Gunakan bahasa Indonesia dan nama kelas yang sama dengan nama berkas.

Format keluaran: tuliskan rencana kerja dan daftar berkas yang akan dibuat. Jangan menulis kode sebelum saya meminta tahap tertentu.

- Bandingkan rencana AI dengan struktur yang dibuat sebeumnya.
- Apabila AI langsung memberikan semua kode, jangan menyalinnya. Kirim ulang kalimat: "Ikuti format keluaran. Berikan rencana dan daftar berkas saja, tanpa kode."
- Hasil yang harus diperiksa: proyek dapat dibuka, folder lib terlihat, dan AI belum menulis kode sebelum diminta.



Gambar 5.6 Rencana kerja proyek Cubit yang dihasilkan oleh AI

b. Membuat *CartCubit* dengan Bantuan AI

Tahap ini meminta AI merancang model data terlebih dahulu, baru menuliskan *KeranjangCubit* secara lengkap.

- Buat folder lib/models, lalu kirim prompt berikut.

Lanjutkan proyek kantin_kampus_cubit_ai ke tahap model data.

Tugas - rancangan lib/models/produk.dart:
1. Usulkan struktur class Produk dengan bidang nama, harga, dan ikon.
2. Usulkan struktur class ItemKeranjang yang menyimpan satu Produk dan jumlah.

3. Jelaskan mengapa kedua class sebaiknya dibuat immutable (memakai const constructor).

Format keluaran: tuliskan rancangan dan alasannya saja. Jangan menulis kode lengkap dahulu.

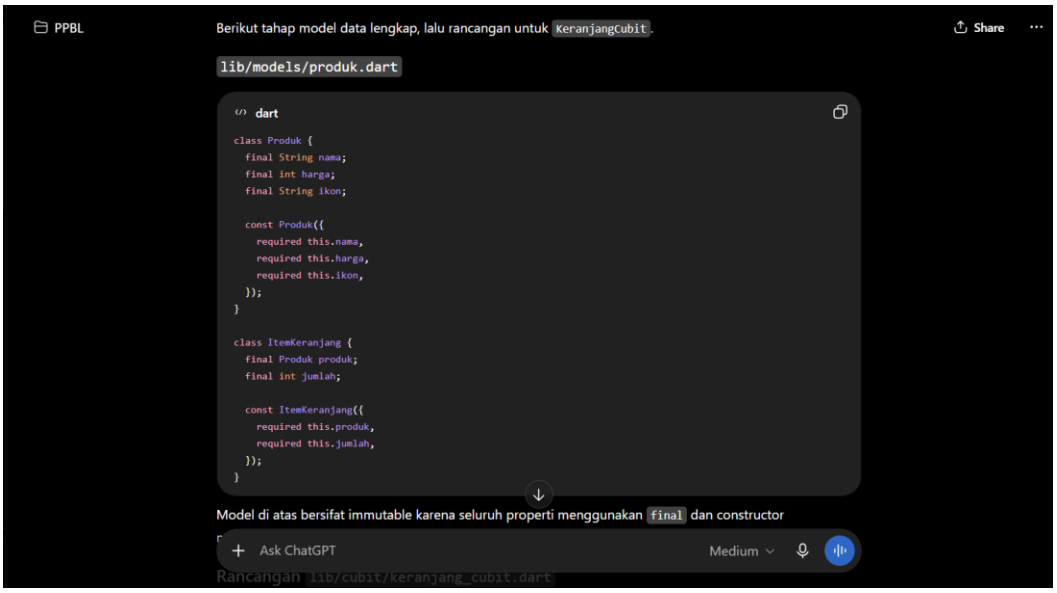
- Bandingkan usulan AI dengan Kode sebelumnya.
- Jika rancangan sudah sesuai, kirim prompt lanjutan berikut untuk melengkapi kode *Produk* dan *ItemKeranjang*, serta merancang *KeranjangCubit*.

Lengkapi lib/models/produk.dart berdasarkan rancangan sebelumnya, lalu lanjutkan ke rancangan lib/cubit/keranjang_cubit.dart.

Tugas:
1. Tuliskan class Produk dan ItemKeranjang secara lengkap sesuai rancangan.
2. Usulkan struktur KeranjangCubit yang meng-extend Cubit<List<ItemKeranjang>>, beserta method tambah(), kurangi(), dan kosongkan().
3. Jelaskan mengapa setiap method harus membuat daftar baru sebelum memanggil emit(), bukan mengubah daftar lama secara langsung.

Batasan: hanya gunakan package:flutter_bloc/flutter_bloc.dart dan berkas produk.dart yang sudah dibuat.

- Salin kode ke berkas yang sesuai, simpan, lalu jalankan flutter analyze untuk memastikan tidak ada galat penulisan.
- Bandingkan struktur kode dengan program D. Nama variabel boleh berbeda, tetapi logika mencari *produk* yang sama dan menambah/mengurangi jumlah harus sama.
- Hasil yang harus diperiksa: berkas *produk*.dart dan keranjang_*cubit*.dart lengkap, setiap method yang mengubah data memanggil *emit*(), dan tidak ada galat saat flutter analyze dijalankan.



Gambar 5.7 Kode KeranjangCubit hasil AI

c. Menghubungkan *Cubit* dengan Bantuan AI

Tahap ini melengkapi proyek kantin_kampus_*cubit*_ai hingga dapat dijalankan penuh: halaman *Produk*, halaman Keranjang, dan main.dart.

- Kirim prompt berikut untuk membuat lib/pages/halaman_*produk*.dart dan lib/pages/halaman_keranjang.dart sekaligus.

```
Lanjutkan ke tahap tampilan.

Tugas - lib/pages/halaman_produk.dart:
1. Tampilkan daftar tiga produk contoh (nama, harga, ikon) memakai ListView dan Card.
2. Setiap kartu memiliki tombol Tambah yang memanggil
context.read<KeranjangCubit>().tambah(produk).

Tugas - lib/pages/halaman_keranjang.dart:
1. Gunakan BlocBuilder<KeranjangCubit, List<ItemKeranjang>> untuk membaca daftar
item.
2. Tampilkan empty state ketika keranjang kosong, daftar item beserta tombol tambah/kurangi
jumlah ketika tidak kosong, dan total harga di bagian bawah.
3. Sediakan tombol Kosongkan Keranjang.

Batasan: kedua berkas hanya boleh mengimpor package:flutter/material.dart,
package:flutter_bloc/flutter_bloc.dart, dan berkas model/cubit yang sudah dibuat.
```

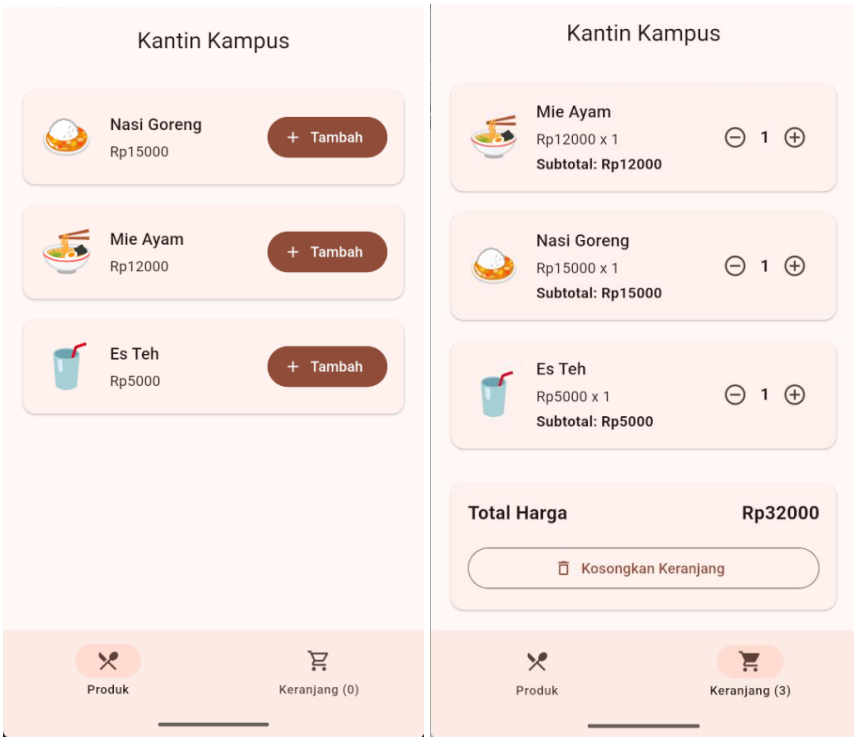
- Salin kedua kode ke berkas yang sesuai, lalu bandingkan strukturnya dengan Kode bagian E.
- Kirim prompt berikut untuk melengkapi lib/main.dart, menggabungkan kedua halaman dengan *NavigationBar* seperti pada Modul 3.

```
Lengkapi lib/main.dart agar menggabungkan halaman Produk dan Keranjang.

Tugas:
1. Bungkus aplikasi dengan BlocProvider, dengan create: (context) => KeranjangCubit().
2. Buat KerangkaUtama dengan NavigationBar dua tujuan: Produk dan Keranjang, memakai
IndexedStack agar kedua halaman tetap hidup di balik layar.
3. Tampilkan jumlah totalItem pada judul tujuan Keranjang, dibaca melalui
context.select<KeranjangCubit, int>.

Batasan: indeks tujuan terpilih tetap memakai setState() sebagai state lokal, bukan Cubit.
```

- Jalankan **flutter run**, lalu uji seluruh alur seperti pada Bagian E: tambah *produk*, pindah ke halaman Keranjang, kurangi/tambah jumlah, dan kosongkan keranjang.
- Hasil yang harus diperiksa: aplikasi berjalan tanpa galat, navigasi antara *Produk* dan Keranjang berfungsi, dan hasilnya identik dengan proyek kantin_kampus_*cubit* yang dibuat secara manual pada Bagian E.



Gambar 5.8 Aplikasi Kantin Kampus (Cubit) hasil AI

d. Membuat *CartBloc* dengan Bantuan AI

Setelah proyek kantin_kampus_*cubit*_ai selesai, buat proyek kedua kantin_kampus_*bloc*_ai dengan cara yang sama seperti pada Bagian a, lalu mulai percakapan AI baru.

- Jalankan flutter create kantin_kampus_*bloc*_ai, tambahkan *flutter_bloc*, lalu kirim ulang prompt konteks awal seperti pada Bagian a, dengan mengganti kata *Cubit* menjadi *BLoC* dan nama proyek menjadi kantin_kampus_*bloc*_ai. Salin kelas *Produk* dan *ItemKeranjang* dari kantin_kampus_*cubit*_ai apa adanya ke lib/models/*produk*.dart.
- Kirim prompt berikut untuk merancang dan menuliskan *KeranjangEvent*, *KeranjangState*, dan *KeranjangBloc*.

```
Lanjutkan proyek kantin_kampus_bloc_ai ke tahap BLoC.

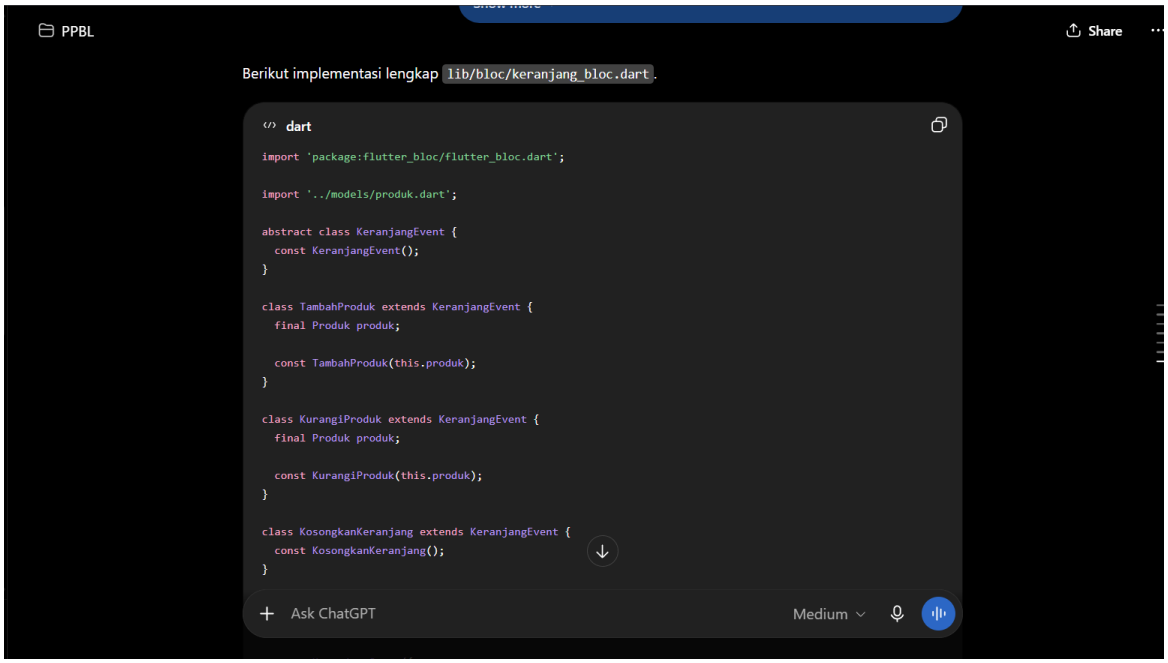
Tugas - lib/bloc/keranjang_bloc.dart:
1. Buat class dasar KeranjangEvent, beserta tiga turunannya: TambahProduk (membawa data Produk), KurangiProduk (membawa data Produk), dan KosongkanKeranjang (tanpa data).
2. Buat class KeranjangState yang membungkus List<ItemKeranjang> items.
3. Buat class KeranjangBloc yang meng-extend Bloc<KeranjangEvent, KeranjangState>, dengan handler on<TambahProduk>, on<KurangiProduk>, dan on<KosongkanKeranjang> yang logikanya setara dengan method pada KeranjangCubit sebelumnya.

Batasan: hanya gunakan package:flutter_bloc/flutter_bloc.dart dan berkas produk.dart yang sudah disalin.

Setelah kode, jelaskan mengapa BLoC memerlukan dua class tambahan (Event dan State) yang tidak diperlukan oleh Cubit.
```

- Salin kode ke berkas yang sesuai, lalu jalankan flutter analyze.

- Bandingkan hasilnya dengan Kode Bagian E.d. Isi setiap handler harus setara dengan isi method tambah(), kurangi(), dan kosongkan() pada *KeranjangCubit*.
- Hasil yang harus diperiksa: berkas keranjang_*bloc*.dart lengkap, setiap handler memanggil *emit*() dengan *KeranjangState* baru, dan tidak ada galat saat flutter analyze dijalankan.



Gambar 5.9 Kode KeranjangBloc hasil AI

e. Menghubungkan *Bloc* dengan Bantuan AI

Tahap ini melengkapi proyek kantin_kampus_*bloc*_ai hingga dapat dijalankan penuh, dengan struktur yang sama seperti proyek *Cubit*.

- Kirim prompt berikut untuk membuat lib/pages/halaman_*produk*.dart dan lib/pages/halaman_keranjang.dart.

```
Lanjutkan ke tahap tampilan pada proyek BLoC.

Tugas - lib/pages/halaman_produk.dart:
1. Tampilkan daftar tiga produk contoh yang sama seperti pada proyek Cubit.
2. Tombol Tambah memanggil context.read<KeranjangBloc>().add(TambahProduk(produk)),
bukan memanggil method secara langsung.

Tugas - lib/pages/halaman_keranjang.dart:
1. Gunakan BlocBuilder<KeranjangBloc, KeranjangState> untuk membaca state.items.
2. Tombol kurangi dan kosongkan mengirim Event KurangiProduk dan KosongkanKeranjang
melalui add().

Batasan: struktur tampilan (empty state, total harga, tombol Kosongkan Keranjang) harus
sama persis dengan proyek Cubit.
```

- Kirim prompt berikut untuk melengkapi lib/main.dart, menggabungkan kedua halaman dengan *NavigationBar*.

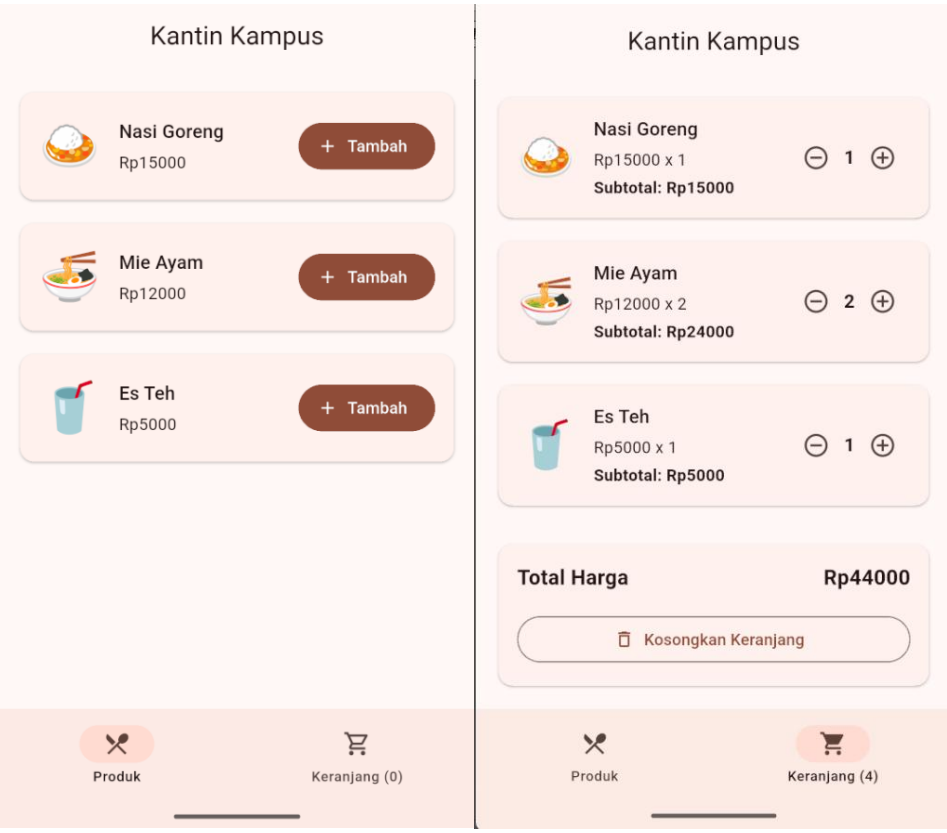
```
Lengkapi lib/main.dart proyek kantin_kampus_bloc_ai.

Tugas:
```

1. Bungkus aplikasi dengan BlocProvider, dengan create: (context) => KeranjangBloc().
2. Buat KerangkaUtama dengan NavigationBar yang sama seperti pada proyek Cubit.
3. Tampilkan jumlah totalItem pada judul tujuan Keranjang, dibaca melalui context.select<KeranjangBloc, int>.

Batasan: struktur KerangkaUtama harus identik dengan proyek Cubit, hanya sumber datanya yang berbeda.

- Jalankan **flutter run**, lalu uji seluruh alur yang sama seperti pada proyek *Cubit*.
- Bandingkan kedua proyek AI (*Cubit* dan *BLoC*) dengan Bagian E. Perbedaan yang wajar hanya pada cara mengirim aksi: method langsung pada *Cubit*, *Event* melalui add() pada *BLoC*.
- Hasil yang harus diperiksa: aplikasi berjalan tanpa galat, hasil di layar identik dengan proyek *Cubit*, dan setiap aksi pengguna benar-benar dikirim sebagai *Event*.



Gambar 5.10 Aplikasi Kantin Kampus (BLoC) hasil AI

f. Menampilkan Efek Samping dengan Bantuan AI

Tahap ini meminta AI menambahkan efek samping (SnackBar) pada proyek kantin_kampus_*bloc*_ai, tanpa mengubah bagian lain yang sudah berjalan.

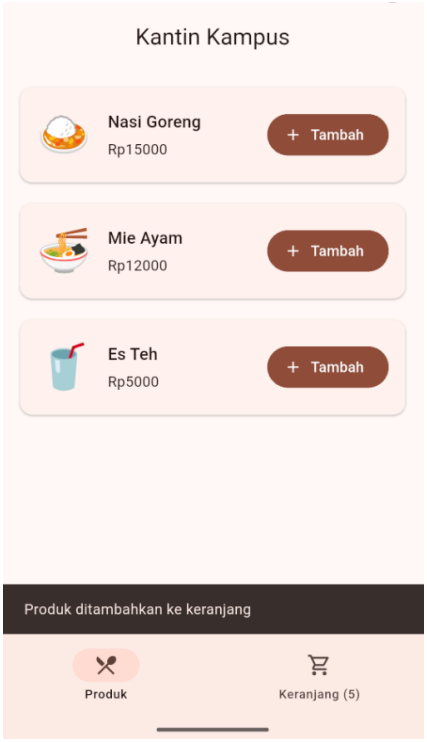
- Kirim prompt berikut pada percakapan proyek *BLoC*.

Perbarui lib/pages/halaman_produk.dart pada proyek kantin_kampus_bloc_ai.

Tugas:
1. Bungkus tampilan yang sudah ada dengan BlocListener<KeranjangBloc, KeranjangState>.
2. Di dalam listener, tampilkan SnackBar bertuliskan "Produk ditambahkan ke keranjang" setiap kali state.items tidak kosong.
3. Jangan mengubah bagian ListView dan tombol Tambah yang sudah ada.

Format keluaran: tampilkan berkas halaman_produk.dart secara lengkap, bukan hanya bagian yang berubah.

- Salin kode ke berkas yang sesuai, lalu jalankan aplikasi dan tekan tombol Tambah.
- Bandingkan hasilnya. Perhatikan apakah AI meletakkan kode SnackBar di listener (benar) atau di builder (keliru, akan membuat SnackBar muncul berulang).
- Hasil yang harus diperiksa: SnackBar muncul tepat satu kali setiap kali tombol Tambah ditekan, dan tampilan daftar *produk* tidak berubah dari sebelumnya.



Gambar 5.11 SnackBar hasil AI pada proyek BLoC

g. Membandingkan Ketiga Pendekatan dengan Bantuan AI

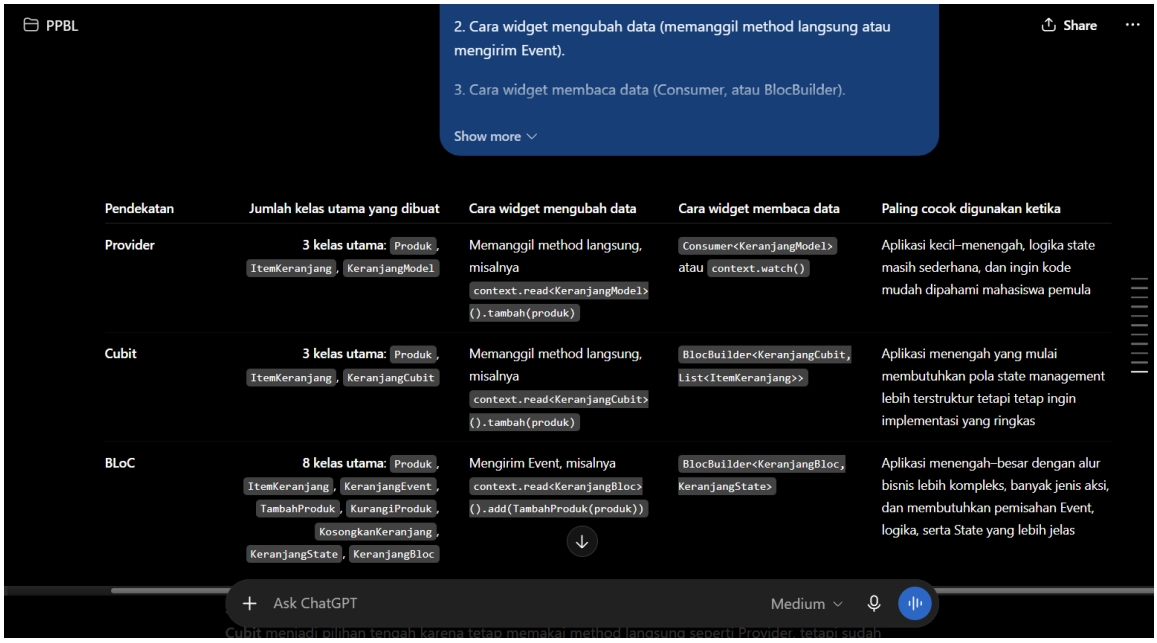
Tahap ini tidak menghasilkan kode baru. AI diminta membantu meringkas perbedaan *Provider*, *Cubit*, dan *BLoC* berdasarkan ketiga proyek yang telah dibuat (Modul 3, kantin_kampus_*cubit*_ai, kantin_kampus_*bloc*_ai), untuk dibandingkan dengan pemahaman mahasiswa sendiri.

- Kirim prompt berikut pada percakapan mana pun, tanpa meminta AI mengubah kode.

Berdasarkan tiga proyek yang telah kita buat (Provider, KeranjangCubit pada kantin_kampus_cubit_ai, dan KeranjangBloc pada kantin_kampus_bloc_ai), buatkan tabel ringkas yang membandingkan:
1. Jumlah kelas yang harus dibuat pada masing-masing pendekatan.
2. Cara widget mengubah data (memanggil method langsung atau mengirim Event).
3. Cara widget membaca data (Consumer, atau BlocBuilder).
4. Situasi yang paling cocok untuk masing-masing pendekatan.

Jangan mengubah kode. Jawab hanya dengan tabel dan penjelasan singkat.

- Bandingkan tabel dari AI dengan Tabel 5.2. Tandai bagian yang berbeda atau kurang tepat, lalu diskusikan pada sesi praktikum.
- Hasil yang harus diperiksa: mahasiswa dapat menjelaskan ulang perbedaan ketiga pendekatan dengan kata-kata sendiri, tanpa membaca ulang jawaban AI.



Gambar 5.12 Hasil perbandingan dari AI

h. Menguji dan Memperbaiki Galat dengan Bantuan AI

Galat (*error*) hampir pasti muncul di salah satu dari kedua proyek AI. Tahap ini melatih cara meminta bantuan AI untuk memperbaiki galat tanpa menulis ulang seluruh berkas.

- Jika muncul galat, catat pesan galat lengkap dari terminal atau panel Problems, lalu kirim prompt berikut tanpa meminta AI menulis ulang seluruh proyek.

Saya mendapatkan galat berikut saat menjalankan proyek Flutter kantin_kampus_cubit_ai (atau kantin_kampus_bloc_ai):

<tempelkan pesan galat lengkap di sini>

Tugas kamu:
1. Jelaskan arti galat dengan bahasa sederhana.
2. Sebutkan nama berkas dan baris yang paling mungkin menjadi penyebab.
3. Tampilkan perbaikan minimal, hanya bagian kode yang berubah.
4. Jelaskan mengapa perbaikan tersebut menyelesaikan galat.

Jangan menulis ulang seluruh berkas dan jangan mengubah fitur lain.

- Terapkan perbaikan secara manual (jangan menempelkan seluruh berkas baru dari AI), lalu jalankan flutter analyze dan flutter run kembali.
- Hasil yang harus diperiksa: kedua proyek (*Cubit* dan *BLoC*) berjalan tanpa galat, navigasi antara *Produk* dan Keranjang berfungsi, dan jumlah item pada judul halaman Keranjang selalu sesuai.

i. Mengevaluasi Jawaban AI

Tahap penutup ini melatih mahasiswa menilai kode dari AI, bukan sekadar menyalinnya.

- Jawab pertanyaan berikut menggunakan kata-kata sendiri: kapan sebaiknya memakai *Cubit* dan kapan memakai *BLoC*; mengapa *BLoC* memerlukan class *Event* terpisah sedangkan *Cubit* tidak; serta mengapa kode SnackBar tidak boleh diletakkan di dalam builder pada *BlocBuilder*.
- Pilih satu bagian kode dari AI (pada proyek *Cubit* maupun *BLoC*) yang belum sepenuhnya dipahami, lalu kirim prompt berikut tanpa meminta AI mengubah kodenya.

> Jelaskan potongan kode Flutter berikut baris demi baris untuk mahasiswa pemula. Jangan mengubah kodenya.
>
> <tempelkan potongan kode yang belum dipahami>
>
> Untuk setiap bagian, jelaskan:
> 1. Apa tugasnya.
> 2. Data apa yang masuk dan keluar.
> 3. Apa yang terjadi jika bagian itu dihapus.
> 4. Apakah bagian itu berhubungan dengan Cubit atau dengan BLoC.

- Hasil yang harus diperiksa: mahasiswa dapat menjelaskan ulang bagian kode yang dipilih dengan kata-kata sendiri, dan dapat menunjukkan perbedaannya dengan pendekatan yang lain (*Provider*, *Cubit*, atau *BLoC*) tanpa membaca ulang jawaban AI.

G. Tugas Praktikum

Soal 1: Menambahkan Manajemen *State* pada Aplikasi Layanan Warga "Nusantara Cerdas Mobile" dengan *Cubit* atau *BLoC*

Aplikasi layanan warga Nusantara Cerdas Mobile yang dibangun pada Modul II dinilai berhasil oleh Dinas Komunikasi dan Informatika. Warga kini dapat berpindah antara tujuan Beranda, Layanan, dan Warga tanpa harus menggulir satu halaman panjang, membuka menu pendukung melalui panel samping, serta menikmati tampilan yang menyesuaikan diri antara layar sempit dan layar lebar di lobi balai kota. Akan tetapi, seluruh data pada aplikasi tersebut masih berupa nilai tetap yang ditulis langsung di dalam kode, sehingga setiap interaksi warga belum benar-benar tersimpan. Kepala dinas menerima banyak keluhan bahwa warga harus mengingat sendiri layanan mana yang pernah mereka cari di antara tiga bidang layanan, dan tidak ada cara untuk mengajukan beberapa layanan sekaligus dalam satu kali kunjungan ke balai kota.

Kepala dinas kemudian meminta tiga peningkatan pada purwarupa yang sudah ada. Pertama, warga harus dapat menandai layanan pada tujuan Layanan sebagai favorit agar mudah ditemukan kembali dari tujuan Warga, tanpa memandang tab bidang Perizinan, Kesehatan, atau Transportasi tempat layanan tersebut berada. Kedua, warga harus dapat memasukkan beberapa layanan ke dalam satu keranjang pengajuan sebelum mengirimkannya sekaligus ke dinas terkait, dan jumlah layanan yang sedang diajukan harus tetap terlihat pada ikon tujuan Warga meskipun warga sedang membuka tujuan Beranda atau Layanan. Ketiga, tombol Ajukan Permohonan pada halaman rincian layanan harus menampilkan tanda sedang mengirim sesaat sebelum permohonan benar-benar tercatat, agar warga tidak menekan tombol tersebut berulang kali karena mengira aplikasi tidak merespons. Petugas balai kota menegaskan bahwa ketiga peningkatan ini tidak

boleh mengubah kerangka navigasi yang sudah berjalan. Sebagai anggota tim pengembang, tugas Anda adalah melanjutkan purwarupa tersebut dengan menambahkan pengelolaan *state* yang sesuai, menggunakan *Cubit* atau *BLoC*.

Ketentuan Pengerjaan

- Lanjutkan proyek nusantara_cerdas_nav_nim yang kerangka navigasinya sudah dibuat pada Modul II. Jika belum pernah dikerjakan, buat ulang kerangka navigasi dasarnya (*NavigationBar*, *NavigationDrawer*, tab navigation, dan *NavigationRail* adaptif) sebelum menambahkan *state*.
- Pilih satu pendekatan untuk seluruh tugas ini: *Cubit* atau *BLoC* (paket *flutter_bloc*). *Provider* dan *ChangeNotifier* tidak boleh dipakai pada tugas ini, kecuali status sedang mengirim pada poin ketujuh yang tetap memakai *setState*() sebagai *state* lokal.
- Buat pengelola *state* untuk data favorit. Jika memilih *Cubit*, buat berkas lib/*cubit*/favorit_*cubit*.dart berisi class *FavoritCubit* yang meng-extend *Cubit*<Set<String>>, menyimpan kumpulan nama layanan favorit, serta method tandai(String namaLayanan) dan batalTandai(String namaLayanan) yang meng-*emit* salinan baru dari kumpulan tersebut. Jika memilih *BLoC*, buat berkas lib/*bloc*/favorit_*bloc*.dart berisi *FavoritEvent* (TandaiLayanan dan BatalTandaiLayanan), *FavoritState*, dan *FavoritBloc* dengan logika yang setara.
- Buat pengelola *state* kedua dengan pendekatan yang sama (konsisten dengan pilihan pada poin sebelumnya) untuk data pengajuan, berisi *PengajuanCubit* atau *PengajuanBloc* yang menyimpan daftar layanan yang sedang diajukan beserta turunan totalPengajuan.
- Sediakan kedua pengelola *state* melalui *BlocProvider* atau *MultiBlocProvider* di akar aplikasi, di atas *MaterialApp* yang sudah memuat named routes dari Modul II.
- Pada tujuan Layanan, setiap kartu layanan diberi ikon bintang yang memanggil method *Cubit* secara langsung atau mengirim *Event* ke *BLoC* (sesuai pendekatan yang dipilih) untuk menandai atau membatalkan tanda, sedangkan warna ikon mengikuti status favorit melalui *BlocBuilder* atau *BlocSelector*.
- Tambahkan bagian Layanan Favorit pada tujuan Warga yang menggunakan *BlocBuilder* agar daftar layanan favorit tampil reaktif, tanpa memandang tab bidang asal layanan tersebut.
- Pada halaman rincian layanan, tombol Ajukan Permohonan menyimpan status sedang mengirim sebagai *state* lokal melalui *setState*(), menampilkan indikator singkat, kemudian menambahkan layanan ke pengelola *state* Pengajuan setelah proses selesai, sambil tetap mengirim nilai balik melalui Navigator.pop(context, nilai) seperti pada Modul II.
- Tampilkan badge jumlah pengajuan pada ikon tujuan Warga menggunakan *BlocSelector* (padanan context.select untuk *Cubit*/*BLoC*) agar hanya bagian badge yang dibangun ulang ketika totalPengajuan berubah, tanpa membangun ulang *NavigationBar* maupun *NavigationRail*.
- Pastikan penempatan *BlocProvider* atau *MultiBlocProvider* tidak menimbulkan galat "*provider* tidak ditemukan" pada layar sempit maupun layar lebar yang sudah dibuat pada Modul II.
- Jalankan aplikasi pada emulator atau perangkat fisik dan pada peramban Google Chrome, tandai beberapa layanan sebagai favorit, ajukan beberapa permohonan, lalu ambil tangkapan layar setiap tahap pada tampilan layar sempit dan layar lebar.

Luaran yang Dikumpulkan

- Kode program lanjutan pada repositori Git.
- Laporan dalam format PDF yang dikumpulkan melalui LMS.
- Tangkapan layar layanan sebelum dan setelah ditandai favorit, bagian Layanan Favorit pada tujuan Warga yang menampilkan daftar reaktif, badge jumlah pengajuan yang berubah, serta indikator sedang mengirim saat tombol Ajukan Permohonan ditekan.
- Penjelasan singkat pada laporan mengenai alasan pemilihan *setState*() untuk status sedang mengirim dan *Cubit* atau *BLoC* untuk data favorit serta pengajuan, termasuk alasan memilih *Cubit* atau *BLoC* dibandingkan yang lain, serta alasan pemilihan *BlocBuilder*, context.read(), atau *BlocSelector*() pada setiap bagian.
- Berkas laporan diberi nama Modul5_NIM_NamaLengkap.pdf dan diunggah melalui LMS sesuai tenggat yang ditentukan.

Soal 2: Menambahkan Manajemen *State* pada Rancangan Tugas Akhir Saudara

Ketentuan Pengerjaan

- Lanjutkan proyek tubes_namaaplikasi_nim yang kerangka navigasinya sudah dibuat pada Soal 2 Modul II.
- Identifikasi minimal satu data yang bersifat lokal dan hanya penting bagi satu halaman atau satu *widget*, misalnya status tampil/sembunyi, indeks tab lokal, atau status memuat, lalu kelola data tersebut dengan *setState*().
- Identifikasi minimal satu data yang dipakai bersama oleh dua halaman atau lebih, misalnya daftar favorit, keranjang, atau status masuk pengguna, lalu buat pengelola *state* dengan *Cubit* atau *BLoC* (pilih salah satu) untuk data tersebut. *Provider* dan *ChangeNotifier* tidak boleh dipakai.
- Sediakan pengelola *state* tersebut melalui *BlocProvider* atau *MultiBlocProvider* di akar aplikasi, di atas *widget* navigasi yang sudah dibuat pada Modul II.
- Tampilkan data dari *Cubit* atau *BLoC* tersebut pada minimal dua halaman berbeda, masing-masing memakai *BlocBuilder*, context.read(), atau *BlocSelector*() sesuai kebutuhan halaman tersebut.
- Data yang dikelola boleh sementara atau contoh sederhana, tetapi perubahan data harus terlihat nyata pada tampilan, bukan nilai tetap.
- Pastikan aplikasi tetap berjalan tanpa galat "*provider* tidak ditemukan" pada seluruh halaman yang mengakses *Cubit* atau *BLoC*.

Luaran yang Dikumpulkan

- Kode program pada repositori Git.
- Tangkapan layar setiap halaman sebelum dan setelah *state* berubah.
- Laporan Modul5_Soal2_NIM_NamaLengkap.pdf diunggah melalui LMS, menjelaskan data mana yang memakai *setState*() dan mana yang memakai *Cubit* atau *BLoC*, beserta alasannya.

H. Format Pengumpulan Tugas

Laporan disusun secara berurutan dengan sistematika berikut.

1. **Halaman Sampul (Cover)**
   Memuat judul praktikum, nama mahasiswa, NIM, kelas, mata kuliah, nama dosen pengampu, program studi, serta tahun akademik. Gunakan logo institusi dan tata letak yang rapi.
2. **Tujuan Praktikum**
   Menuliskan tujuan yang ingin dicapai pada praktikum ini dengan kalimat sendiri, bukan menyalin ulang tujuan pembelajaran pada modul.
3. **Alat dan Bahan**
   Menyebutkan spesifikasi perangkat keras yang digunakan serta perangkat lunak beserta versinya, misalnya Flutter SDK, Dart SDK, Android Studio, Visual Studio Code, dan Git.
4. **Langkah-Langkah Praktikum**
   Menguraikan tahapan pengerjaan secara runtut mulai dari pembuatan proyek, pembuatan stateless *widget*, stateful *widget*, penyusunan layout dan styling, hingga penggabungan pada main.dart. Setiap langkah disertai potongan kode yang relevan dan penjelasan singkat.
5. **Hasil dan Dokumentasi Program**
   Menampilkan tangkapan layar hasil aplikasi yang berjalan, sekurang-kurangnya tampilan awal dan tampilan setelah terjadi perubahan *state*. Setiap gambar diberi nomor, keterangan gambar, dan penjelasan singkat.
6. **Analisis dan Pembahasan**
   Membahas alasan pemilihan stateless *widget* dan stateful *widget* pada masing-masing bagian aplikasi, kendala atau galat yang ditemukan beserta cara penyelesaiannya, serta keterkaitan aplikasi dengan pilar smart city yang diangkat.
7. **Kesimpulan**
   Menuliskan ringkasan capaian praktikum dan pemahaman yang diperoleh, disusun dalam beberapa kalimat yang menjawab tujuan praktikum.
8. **Lampiran berisi Link Repositori GitHub**
   Mencantumkan tautan repositori GitHub yang dapat diakses oleh dosen pengampu, memuat kode program lengkap.
9. **Presentasi Hasil Praktikum**
   Mahasiswa mempresentasikan hasil praktikum di kelas, meliputi demonstrasi aplikasi yang berjalan dan penjelasan kode program, serta menjawab pertanyaan dari dosen pengampu.